

Documentação do Projeto BarberPro

1. Introdução

O projeto **BarberPro** é uma aplicação web completa desenvolvida para gerenciar barbearias, oferecendo funcionalidades para clientes agendarem serviços, barbeiros gerenciarem suas agendas e gerentes administrarem a barbearia. A aplicação é construída com uma arquitetura de microsserviços, utilizando C#/.NET para o backend, React/TypeScript para o frontend e PostgreSQL como banco de dados.

Este documento visa fornecer uma visão abrangente da arquitetura, componentes e funcionalidades do sistema, servindo como um guia para desenvolvedores, administradores e qualquer pessoa interessada em entender o funcionamento interno do BarberPro.

2. Estrutura do Projeto

O repositório do BarberPro é organizado em três diretórios principais, cada um representando uma parte fundamental da aplicação:

- **Backend/** : Contém todo o código-fonte da API RESTful desenvolvida em C# com .NET. É responsável pela lógica de negócios, interação com o banco de dados e autenticação.
- **Frontend/** : Abriga o código-fonte da interface do usuário, construída com React e TypeScript. Esta parte da aplicação é responsável por interagir com o usuário e consumir os serviços fornecidos pelo backend.
- **Database/** : Inclui os scripts SQL para a criação e inicialização do banco de dados PostgreSQL, definindo o esquema das tabelas e as relações entre elas.

Essa separação clara de responsabilidades facilita o desenvolvimento, a manutenção e a escalabilidade do projeto, permitindo que cada componente seja desenvolvido e implantado de forma independente.

3. Backend (C#/.NET)

O backend do BarberPro é uma API RESTful desenvolvida em C# utilizando o framework .NET. Ele segue uma arquitetura modular, com a lógica de negócios bem definida e separada em diferentes camadas.

3.1. Visão Geral da Arquitetura

A arquitetura do backend é baseada no padrão Model-View-Controller (MVC) para APIs, embora com algumas adaptações para o contexto de uma API REST. Os principais componentes são:

- **Controllers**: Responsáveis por receber as requisições HTTP, processá-las e retornar as respostas. Eles atuam como a camada de interface entre o cliente (frontend) e a lógica de negócios.
- **Models**: Representam as entidades de dados do sistema (ex: `Usuario`, `Barbearia`, `Agendamento`). São classes C# que mapeiam as tabelas do banco de dados.

- **Services:** Contêm a lógica de negócios principal da aplicação. São responsáveis por orquestrar as operações, interagir com o banco de dados (através do `DbContext`) e implementar regras de negócio complexas. A injeção de dependência é amplamente utilizada para gerenciar essas dependências.
- **Data:** Inclui o `DbContext` (Entity Framework Core) e as classes de configuração para o acesso ao banco de dados. É a camada de persistência de dados.
- **DTOs (Data Transfer Objects):** Classes utilizadas para transferir dados entre as camadas da aplicação, especialmente entre os controladores e os serviços, e para definir o formato dos dados de entrada e saída das APIs.

3.2. Componentes Principais

3.2.1. Program.cs

O arquivo `Program.cs` é o ponto de entrada da aplicação backend. Ele é responsável por configurar o host da aplicação, registrar os serviços no contêiner de injeção de dependência e configurar o pipeline de middlewares HTTP. As principais configurações incluem:

- **Configuração de URLs:** A aplicação é configurada para escutar em todas as interfaces de rede (`0.0.0.0`) e utiliza a variável de ambiente `PORT` para definir a porta, com `5000` como padrão. Isso é crucial para implantações em ambientes de nuvem como o Render.com.
- **Entity Framework Core:** O `BarbeariaContext` é configurado para usar PostgreSQL, com a string de conexão obtida da variável de ambiente `DATABASE_URL` ou do `appsettings.json`. As migrações do banco de dados são aplicadas automaticamente na inicialização da aplicação, garantindo que o esquema do banco esteja sempre atualizado.
- **Autenticação JWT:** A autenticação baseada em JSON Web Tokens (JWT) é configurada. Isso permite que os usuários se autenticem uma vez e recebam um token que pode ser usado para acessar recursos protegidos em requisições subsequentes. Os parâmetros de validação do token (emissor, audiência, chave de assinatura) são definidos usando configurações do aplicativo.
- **Serviços Personalizados:** Serviços como `IAuthService` (para hash de senha, geração de token) e `IGoogleAuthService` (para autenticação via Google) são registrados para injeção de dependência. Isso promove a separação de preocupações e facilita a testabilidade.
- **CORS (Cross-Origin Resource Sharing):** Uma política de CORS (`AllowSpecificOrigin`) é configurada para permitir requisições de origens específicas (como o frontend hospedado no Netlify ou em ambientes de desenvolvimento local). Isso é essencial para que o frontend, que geralmente roda em um domínio diferente, possa se comunicar com o backend.
- **Swagger/OpenAPI:** Em ambiente de desenvolvimento, o Swagger é habilitado para fornecer documentação interativa da API, facilitando o teste e o entendimento dos endpoints disponíveis.
- **Middlewares:** O pipeline de requisições inclui middlewares para CORS, roteamento, autenticação, autorização e servir arquivos estáticos.

3.2.2. Controllers

Os controladores são classes que herdam de `ControllerBase` e contêm métodos de ação que respondem a requisições HTTP. Eles são decorados com `[ApiController]` e `[Route]` para definir o comportamento da API. Exemplos de controladores incluem:

- **AuthController.cs** : Gerencia todas as operações relacionadas à autenticação e registro de usuários. Isso inclui login, cadastro de clientes, barbeiros e gerentes, cadastro de barbearias (que também cria um gerente inicial), autenticação via Google, e funcionalidades de recuperação/redefinição de senha. Ele utiliza `IAuthService` e `IGoogleAuthService` para a lógica de autenticação e interage com o `BarbeariaContext` para persistência de dados. As validações de entrada são realizadas diretamente nos métodos do controlador, retornando `BadRequest` em caso de falha.
- **AgendamentoController.cs** : Lida com a criação, leitura, atualização e exclusão de agendamentos.
- **BarbeiroController.cs** : Gerencia operações relacionadas aos barbeiros, como listagem, detalhes e atualização de perfil.
- **BarbeariaController.cs** : Permite a gestão de informações da barbearia, como serviços e horários.

3.2.3. Models

As classes de modelo representam as entidades de domínio da aplicação e são mapeadas para as tabelas do banco de dados pelo Entity Framework Core. Elas contêm propriedades que correspondem às colunas da tabela e podem incluir anotações de dados (`[Key]` , `[Required]` , `[StringLength]` , `[ForeignKey]`) para configurar o mapeamento e as validações. Exemplos:

- **Usuario.cs** : Define a estrutura de um usuário, incluindo nome, email, hash de senha, tipo de usuário (Cliente, Barbeiro, Gerente), e relacionamentos com `Barbearia` . Possui campos opcionais para `GoogleId` , `Foto` , `Especialidades` e `Descricao` para acomodar diferentes tipos de usuários.
- **Barbearia.cs** : Representa uma barbearia, com propriedades como nome, endereço, telefone, email, logo, e códigos únicos (`CodigoConvite` , `CodigoBarbearia`). Inclui coleções virtuais para `Usuarios` , `Agendamentos` e `Servicos` , estabelecendo os relacionamentos um-para-muitos.
- **Agendamento.cs** : Descreve um agendamento, com detalhes como cliente, barbeiro, barbearia, data/hora, tipo de serviço, preço, observações e status. Utiliza um `enum` para o `StatusAgendamento` .
- **HorarioDisponivel.cs** : Representa um slot de horário que um barbeiro tem disponível, associado a um `BarbeiroId` e uma `DataHora` .
- **Servico.cs** : Define um serviço oferecido por uma barbearia, incluindo nome, preço e duração, e está associado a uma `BarbeariaId` .

3.2.4. Services

A camada de serviços contém a lógica de negócios e abstrai a interação direta com o `DbContext` . Isso permite que os controladores sejam mais enxutos e focados apenas no tratamento de requisições HTTP. Exemplos:

- **AuthService.cs** : Implementa a lógica de autenticação, como hashing e verificação de senhas (usando `BCrypt`), geração de tokens JWT, e geração de códigos únicos para barbearias e convites. É a implementação da interface `IAuthService` .
- **GoogleAuthService.cs** : Responsável por verificar tokens de autenticação do Google e obter informações do usuário a partir deles. Implementa a interface `IGoogleAuthService` .

3.3. Autenticação

O backend do BarberPro suporta dois métodos de autenticação:

- **Autenticação Baseada em Senha (JWT):** Usuários podem se registrar com email e senha. As senhas são armazenadas como hashes (usando BCrypt) para segurança. Após o login bem-sucedido, um JSON Web Token (JWT) é emitido. Este token deve ser incluído no cabeçalho `Authorization` das requisições subsequentes para acessar rotas protegidas.
- **Autenticação via Google:** Usuários podem se autenticar usando suas contas Google. O frontend envia um `id_token` do Google para o backend, que é verificado usando o `GoogleAuthService`. Se o usuário Google já existir no sistema, ele é logado; caso contrário, um novo usuário é criado com base nas informações do Google.

3.4. Configuração do Banco de Dados

O projeto utiliza **PostgreSQL** como sistema de gerenciamento de banco de dados relacional e **Entity Framework Core** como ORM (Object-Relational Mapper). A configuração é feita no `Program.cs`, onde o `BarbeariaContext` é configurado para usar o provedor `Npgsql`. A string de conexão é flexível, permitindo o uso de variáveis de ambiente para ambientes de produção.

As migrações do Entity Framework Core são aplicadas automaticamente na inicialização, garantindo que o esquema do banco de dados esteja sempre sincronizado com os modelos C#.

4. Frontend (React/TypeScript)

O frontend do BarberPro é uma Single Page Application (SPA) desenvolvida com React e TypeScript, utilizando Vite para o ambiente de desenvolvimento e build. A interface é projetada para ser responsiva e intuitiva, atendendo às necessidades de clientes, barbeiros e gerentes.

4.1. Visão Geral da Arquitetura

A arquitetura do frontend é baseada em componentes React, com uma estrutura de diretórios clara para organizar diferentes partes da aplicação:

- `src/` : Contém todo o código-fonte da aplicação React.
 - `components/` : Componentes React reutilizáveis (ex: `Layout`, `AuthForm`).
 - `pages/` : Componentes que representam páginas completas da aplicação, organizados por tipo de usuário (`client`, `barber`, `manager`).
 - `contexts/` : Contextos React para gerenciamento de estado global (ex: `AuthContext`, `ThemeContext`).
 - `services/` : Funções e classes para interagir com a API do backend.
 - `types/` : Definições de tipos TypeScript para as entidades de dados e DTOs.

4.2. Componentes Principais

4.2.1. `App.tsx`

O arquivo `App.tsx` é o componente raiz da aplicação React. Ele é responsável por:

- **Configuração de Provedores:** Envolve a aplicação com `ThemeProvider` (para gerenciamento de tema) e `AuthProvider` (para gerenciamento de autenticação). Isso torna o tema e os dados do usuário globalmente acessíveis a todos os componentes filhos.
- **Roteamento:** Utiliza `react-router-dom` para definir as rotas da aplicação. O `HashRouter` é empregado para compatibilidade com hospedagem estática (como Netlify), onde o roteamento baseado em histórico pode exigir configurações de servidor.
- **Rotas Protegidas (`ProtectedRoute`):** Um componente `ProtectedRoute` é implementado para controlar o acesso às rotas com base no status de autenticação do usuário e seu tipo (`role`). Se um usuário não estiver autenticado ou não tiver a `role` permitida para uma rota específica, ele é redirecionado para a página de autenticação ou para a dashboard de sua `role` .
- **Estrutura de Layout:** A rota principal (`/`) utiliza um componente `Layout` que serve como um invólucro para as páginas protegidas, fornecendo elementos comuns como navegação e cabeçalho/rodapé.
- **Notificações (`Toaster`):** O `react-hot-toast` é configurado para exibir mensagens de notificação ao usuário de forma amigável.

4.2.2. Contextos

- **`AuthContext.tsx` :** Gerencia o estado de autenticação do usuário. Ele armazena informações como o usuário logado, seu token JWT e funções para login, logout e registro. Este contexto é crucial para manter o estado de autenticação consistente em toda a aplicação e para proteger rotas.
- **`ThemeContext.tsx` :** Provavelmente gerencia o tema da aplicação (claro/escuro), permitindo que os componentes acessem e alterem o tema globalmente.

4.2.3. Páginas

As páginas são componentes React que representam telas completas da aplicação, organizadas por tipo de usuário para clareza e manutenção. Exemplos:

- **`client/` :** Contém páginas para clientes, como `ClientDashboard` (visão geral do cliente), `Barbershops` (listagem de barbearias), `Appointments` (agendamentos do cliente) e `BookAppointment` (tela para agendar um serviço).
- **`barber/` :** Contém páginas para barbeiros, como `BarberDashboard` (visão geral do barbeiro), `BarberSchedule` (agenda do barbeiro), `BarberStats` (estatísticas de desempenho) e `BarberSettings` (configurações de perfil).
- **`manager/` :** Contém páginas para gerentes, como `ManagerDashboard` (visão geral da barbearia), `ManagerBarbers` (gestão de barbeiros), `ManagerStats` (estatísticas da barbearia) e `ManagerSettings` (configurações da barbearia).

4.3. Interação com o Backend

O frontend se comunica com o backend através de requisições HTTP (provavelmente usando `fetch` ou uma biblioteca como `axios`). As funções para essas interações são encapsuladas no diretório `services/` para manter a lógica de comunicação separada dos componentes da UI. O token JWT, obtido no login, é incluído no cabeçalho `Authorization` das requisições para endpoints protegidos.

5. Database (PostgreSQL)

O diretório `Database/` contém o script `init.sql`, que é responsável por configurar o esquema do banco de dados PostgreSQL para o BarberPro. Este script define as tabelas, seus campos, tipos de dados, chaves primárias e estrangeiras, índices, e também inclui funções e triggers para automatizar certas operações.

5.1. Esquema do Banco de Dados

O banco de dados é composto pelas seguintes tabelas principais:

- **barbearias**: Armazena informações sobre cada barbearia, como nome, endereço, telefone, email, logo, e dois códigos únicos: `codigo_convite` (usado para novos barbeiros/gerentes se associarem) e `codigo_barbearia` (identificador único da barbearia).
- **usuarios**: Contém os dados de todos os usuários do sistema (clientes, barbeiros e gerentes). Inclui campos para nome, email, hash da senha, tipo de usuário (definido por um `INTEGER` com `CHECK` constraint), e uma chave estrangeira para `barbearias` (para barbeiros e gerentes). Campos adicionais como `foto`, `especialidades` e `descricao` são específicos para barbeiros.
- **horarios_disponiveis**: Registra os slots de tempo que cada barbeiro tem disponível para agendamentos. Cada registro inclui a `data_hora`, o `barbeiro_id` e um status `esta_disponivel`.
- **agendamentos**: Armazena os detalhes de cada agendamento realizado, incluindo `cliente_id`, `barbeiro_id`, `barbearia_id`, `data_hora`, `tipo_servico`, `preco_servico`, `observacoes` e `status` (definido por um `INTEGER` com `CHECK` constraint).
- **servicos**: Lista os serviços que uma barbearia oferece, com `nome`, `preco`, `duracao_minutos` e uma chave estrangeira para `barbearia_id`.

5.2. Funções e Triggers

O script `init.sql` também define:

- **gerar_codigo_convite()**: Uma função PL/pgSQL que gera um código alfanumérico único de 8 caracteres para o `codigo_convite` de uma nova barbearia. Ela garante a unicidade do código através de um loop que verifica a existência do código gerado antes de retorná-lo.
- **trigger_gerar_codigo_convite()**: Uma função de trigger que chama `gerar_codigo_convite()` automaticamente antes de uma nova linha ser inserida na tabela `barbearias`, preenchendo o campo `codigo_convite` se ele não for fornecido.
- **trigger_atualizar_data_atualizacao()**: Uma função de trigger que atualiza automaticamente a coluna `data_atualizacao` para o `CURRENT_TIMESTAMP` sempre que um registro na tabela `agendamentos` é modificado.

5.3. Índices

Diversos índices são criados para otimizar o desempenho das consultas, especialmente em campos frequentemente usados em cláusulas `WHERE` ou `JOIN`, como emails de usuários, IDs de barbearias e datas de agendamento/horários disponíveis.

5.4. Dados de Exemplo

O script inclui seções comentadas para inserção de dados de exemplo (`INSERT INTO ... ON CONFLICT DO NOTHING`), que podem ser descomentadas para popular o banco de dados com uma barbearia, um gerente, um barbeiro e um cliente de demonstração. Isso é útil para testes e desenvolvimento inicial.

6. Instalação e Configuração

Para configurar e rodar o projeto BarberPro localmente, siga os passos abaixo:

6.1. Pré-requisitos

Certifique-se de ter os seguintes softwares instalados em sua máquina:

- **Git**: Para clonar o repositório.
- **.NET SDK (versão 6.0 ou superior)**: Para compilar e rodar o backend C#.
- **Node.js e npm (ou Yarn/pnpm)**: Para gerenciar as dependências e rodar o frontend React.
- **PostgreSQL**: O servidor de banco de dados. Você pode instalá-lo diretamente ou usar Docker.
- **Um editor de código** (ex: VS Code) com suporte a C#, TypeScript e React.

6.2. Configuração do Banco de Dados

1. **Crie um banco de dados PostgreSQL**: Abra seu cliente PostgreSQL (pgAdmin, psql, DBeaver, etc.) e crie um novo banco de dados, por exemplo, `barbearia_saas`. `sql CREATE DATABASE barbearia_saas;`
2. **Execute o script `init.sql`**: Conecte-se ao banco de dados recém-criado e execute todo o conteúdo do arquivo `Database/init.sql`. Este script criará as tabelas, funções e triggers necessários. `bash psql -U seu_usuario -d barbearia_saas -f /caminho/para/BarberPro/Database/init.sql`
3. **Configure a string de conexão**: No diretório `Backend/`, localize o arquivo `appsettings.json` (ou `appsettings.Development.json` para desenvolvimento) e configure a string de conexão para o seu banco de dados PostgreSQL. Alternativamente, você pode definir a variável de ambiente `DATABASE_URL`. Exemplo em `appsettings.Development.json`: `json { "ConnectionStrings": { "DefaultConnection": "Host=localhost;Port=5432;Database=barbearia_saas;Username=seu_usuario;Password=sua_senha" }, "Jwt": { "Key": "SuaChaveSecretaMuitoLongaParaJWTQueDeveTerPeloMenos32Caracteres", "Issuer": "BarberPro", "Audience": "BarberProUsers" }, "GoogleAuth": { "ClientId": "SEU_CLIENT_ID_DO_GOOGLE" } }` **Importante**: A `Jwt:Key` deve ser uma string longa e segura. O `GoogleAuth:ClientId` deve ser o ID do cliente da sua aplicação Google Cloud para autenticação.

6.3. Rodando o Backend

1. **Navegue até o diretório do backend**: `bash cd /caminho/para/BarberPro/Backend`
2. **Restaurar as dependências**: `bash dotnet restore`
3. **Rode a aplicação**: `bash dotnet run` O backend será iniciado, geralmente na porta 5000 (ou na porta definida pela variável de ambiente `PORT`). Você verá mensagens no console indicando que a aplicação

está rodando.

6.4. Rodando o Frontend

1. **Navegue até o diretório do frontend:** `bash cd /caminho/para/BarberPro/Frontend`
2. **Instale as dependências:** `bash npm install # ou yarn install / pnpm install`
3. **Rode a aplicação em modo de desenvolvimento:** `bash npm run dev # ou yarn dev / pnpm dev` O frontend será iniciado, geralmente na porta 5173. Abra seu navegador e acesse `http://localhost:5173` (ou a URL indicada no console) para ver a aplicação em funcionamento.

7. Considerações Finais

O projeto BarberPro oferece uma solução robusta e escalável para o gerenciamento de barbearias, com uma arquitetura bem definida e o uso de tecnologias modernas. A separação entre frontend, backend e banco de dados permite o desenvolvimento e a manutenção independentes de cada camada, facilitando a colaboração e a evolução do sistema.

Para futuras melhorias, pode-se considerar a implementação de testes unitários e de integração mais abrangentes, a adição de funcionalidades de notificação em tempo real (websockets), e a exploração de ferramentas de CI/CD para automação do deploy. A documentação será atualizada conforme o projeto evoluir.

8. Usabilidade

A usabilidade do BarberPro foi projetada com foco em três perfis de usuários distintos: Clientes, Barbeiros e Gerentes. Cada interface foi cuidadosamente elaborada para ser intuitiva e eficiente, minimizando a curva de aprendizado e otimizando as tarefas diárias de cada grupo.

8.1. Experiência do Cliente

Para os clientes, a prioridade é a facilidade e rapidez no agendamento de serviços. A interface do cliente oferece:

- **Navegação Simplificada:** Menus claros e um fluxo de agendamento passo a passo que guia o usuário desde a seleção da barbearia até a confirmação do serviço.
- **Busca e Filtragem:** Capacidade de buscar barbearias por localização, serviços oferecidos e disponibilidade de barbeiros, permitindo que o cliente encontre rapidamente o que precisa.
- **Visualização de Agenda:** Uma visão clara dos horários disponíveis dos barbeiros, facilitando a escolha do melhor momento para o agendamento.
- **Confirmação e Lembretes:** Após o agendamento, o cliente recebe confirmações e lembretes (futuramente, via notificações ou e-mail/SMS) para evitar esquecimentos.
- **Gestão de Agendamentos:** Acesso fácil para visualizar, reagendar ou cancelar agendamentos existentes, proporcionando flexibilidade ao usuário.

8.2. Experiência do Barbeiro

Os barbeiros precisam de ferramentas que os ajudem a gerenciar sua agenda e serviços de forma eficaz. A interface do barbeiro inclui:

- **Agenda Interativa:** Uma visualização clara e editável dos seus horários, permitindo que o barbeiro adicione sua disponibilidade, visualize agendamentos confirmados e gerencie bloqueios de tempo.
- **Gestão de Serviços:** Capacidade de definir e atualizar os serviços que oferecem, incluindo preços e duração, garantindo que suas ofertas estejam sempre atualizadas.
- **Perfil Personalizável:** Opções para atualizar informações de perfil, como especialidades, descrição e foto, ajudando a atrair mais clientes.
- **Visão Geral de Desempenho:** Acesso a estatísticas básicas sobre seus agendamentos e faturamento (futuramente), auxiliando no acompanhamento de seu desempenho.

8.3. Experiência do Gerente

Para os gerentes, a usabilidade se traduz em controle e visibilidade sobre as operações da barbearia. A interface do gerente oferece:

- **Dashboard Centralizado:** Uma visão consolidada das principais métricas da barbearia, como número de agendamentos, faturamento e desempenho dos barbeiros.
- **Gestão de Equipe:** Ferramentas para adicionar, remover e gerenciar perfis de barbeiros, incluindo suas especialidades e horários.
- **Gestão de Serviços da Barbearia:** Capacidade de configurar e gerenciar todos os serviços oferecidos pela barbearia, garantindo consistência e padronização.
- **Relatórios e Análises:** Acesso a relatórios detalhados sobre agendamentos, clientes e desempenho financeiro (futuramente), fornecendo insights para tomadas de decisão estratégicas.
- **Configurações da Barbearia:** Opções para gerenciar informações da barbearia, como endereço, telefone, email e logo, além de códigos de convite para novos membros da equipe.

Em resumo, a usabilidade do BarberPro é um pilar fundamental, buscando oferecer uma experiência fluida e eficiente para todos os tipos de usuários, adaptando as funcionalidades às suas necessidades específicas e rotinas diárias.

9. Método de Desenvolvimento

O desenvolvimento do BarberPro seguiu uma abordagem que prioriza a modularidade, escalabilidade e manutenibilidade, utilizando tecnologias modernas e padrões de arquitetura bem estabelecidos. A escolha das tecnologias e a estrutura do projeto refletem a intenção de criar uma aplicação robusta e de fácil evolução.

9.1. Abordagem e Filosofia

O projeto foi concebido com uma filosofia de **microserviços**, embora em uma escala inicial que pode ser caracterizada como uma **arquitetura em camadas** com forte separação de responsabilidades. Isso significa que o backend e o frontend são aplicações independentes que se comunicam através de APIs RESTful,

permitindo que sejam desenvolvidos, testados e implantados de forma autônoma. Essa abordagem oferece diversas vantagens:

- **Escalabilidade Independente:** O backend e o frontend podem ser escalados separadamente de acordo com a demanda, otimizando o uso de recursos.
- **Manutenibilidade Aprimorada:** Alterações em uma camada (ex: backend) têm impacto mínimo na outra (frontend), facilitando a manutenção e a introdução de novas funcionalidades.
- **Flexibilidade Tecnológica:** Permite a escolha das melhores tecnologias para cada parte da aplicação, sem amarrar todo o sistema a uma única stack.
- **Desenvolvimento Paralelo:** Equipes diferentes podem trabalhar simultaneamente no frontend e no backend, acelerando o processo de desenvolvimento.

9.2. Escolha de Tecnologias

A seleção das tecnologias foi baseada em sua robustez, popularidade, ecossistema e capacidade de atender aos requisitos do projeto:

- **Backend: C# e .NET:** O .NET é um framework maduro e performático, ideal para construir APIs robustas e escaláveis. A linguagem C# oferece forte tipagem, recursos modernos e um vasto ecossistema de bibliotecas e ferramentas. O uso do Entity Framework Core simplifica a interação com o banco de dados, abstraindo grande parte da complexidade do SQL.
- **Frontend: React e TypeScript:** React é uma biblioteca JavaScript amplamente adotada para construir interfaces de usuário interativas e reativas. A escolha do TypeScript adiciona tipagem estática ao JavaScript, o que melhora a qualidade do código, facilita a detecção de erros em tempo de desenvolvimento e aprimora a manutenibilidade de grandes bases de código. O Vite foi escolhido como ferramenta de build devido à sua velocidade e eficiência no desenvolvimento.
- **Banco de Dados: PostgreSQL:** Um sistema de gerenciamento de banco de dados relacional (SGBDR) de código aberto, conhecido por sua confiabilidade, robustez, conformidade com padrões SQL e extensibilidade. É uma escolha sólida para aplicações que exigem integridade de dados e suporte a transações complexas.

9.3. Padrões de Projeto e Boas Práticas

Durante o desenvolvimento, foram aplicados diversos padrões de projeto e boas práticas para garantir a qualidade e a organização do código:

- **Injeção de Dependência (DI):** Amplamente utilizada no backend para gerenciar as dependências entre os componentes. Isso promove a inversão de controle, facilita a testabilidade (mocking de dependências) e torna o código mais modular e flexível.
- **Padrão Repositório (implícito):** Embora não haja uma camada de repositório explícita separada, a interação com o `DbContext` dentro dos serviços segue os princípios do padrão repositório, abstraindo a lógica de acesso a dados dos controladores.
- **DTOs (Data Transfer Objects):** Utilizados para definir o formato dos dados que transitam entre as camadas da aplicação e entre o frontend e o backend. Isso garante que apenas os dados necessários sejam expostos e que a comunicação seja clara e tipada.
- **Validação de Dados:** As validações são realizadas tanto no frontend (para feedback imediato ao usuário) quanto no backend (para garantir a integridade dos dados antes da persistência). No backend, são

utilizadas anotações de dados e lógica de validação explícita nos controladores.

- **Tratamento de Erros:** Erros são capturados e tratados de forma consistente, retornando respostas HTTP apropriadas (ex: 400 Bad Request, 401 Unauthorized, 500 Internal Server Error) com mensagens claras para o cliente.
- **Segurança:** Senhas são armazenadas como hashes (BCrypt) e a autenticação é feita via JWT, garantindo que as credenciais dos usuários sejam protegidas. A configuração de CORS restringe o acesso à API apenas a origens permitidas.
- **Migrações de Banco de Dados:** O uso do Entity Framework Core para migrações automatiza a evolução do esquema do banco de dados, facilitando a implantação e a manutenção em diferentes ambientes.

9.4. Ferramentas e Ambiente de Desenvolvimento

- **Git:** Controle de versão para gerenciar o histórico do código e facilitar a colaboração.
- **Visual Studio Code:** Editor de código principal, com extensões para C#, React, TypeScript e PostgreSQL.
- **npm/Yarn/pnpm:** Gerenciadores de pacotes para o frontend.
- **Vite:** Ferramenta de build para o frontend, oferecendo um ambiente de desenvolvimento rápido.
- **Postman/Swagger UI:** Utilizados para testar e documentar a API do backend.

Este método de desenvolvimento visa garantir que o BarberPro seja uma aplicação robusta, segura, escalável e fácil de manter, capaz de se adaptar às futuras necessidades do negócio.

10. Pontos Importantes

Ao analisar o projeto BarberPro, diversos pontos se destacam como cruciais para o seu funcionamento, segurança e potencial de expansão. Compreender esses aspectos é fundamental para qualquer intervenção ou evolução do sistema.

10.1. Arquitetura Modular e Separação de Responsabilidades

Um dos pilares do BarberPro é a sua arquitetura bem definida, dividida em três componentes principais: Backend, Frontend e Database. Essa modularidade não é apenas uma questão de organização de código, mas uma decisão arquitetural que traz benefícios significativos:

- **Desenvolvimento Paralelo:** Permite que equipes ou desenvolvedores trabalhem simultaneamente em diferentes partes do sistema sem grandes conflitos, acelerando o ciclo de desenvolvimento.
- **Escalabilidade Independente:** Cada componente pode ser escalado de forma autônoma. Se o backend precisar de mais recursos devido ao aumento de requisições, ele pode ser escalado sem afetar o frontend ou o banco de dados, e vice-versa. Isso é vital para aplicações SaaS (Software as a Service) que precisam se adaptar a diferentes volumes de usuários.
- **Manutenibilidade e Testabilidade:** A separação clara de responsabilidades facilita a identificação e correção de bugs, bem como a implementação de novas funcionalidades. Testes unitários e de integração podem ser focados em módulos específicos, reduzindo a complexidade.
- **Flexibilidade Tecnológica:** Embora o projeto utilize C#/.NET e React/TypeScript, a arquitetura permite que, no futuro, um componente seja reescrito em outra tecnologia, se necessário, sem impactar

drasticamente os demais. Por exemplo, o frontend poderia ser migrado para Vue.js ou Angular sem a necessidade de alterar o backend.

10.2. Segurança na Autenticação e Autorização

A segurança é um aspecto crítico em qualquer aplicação que lida com dados de usuários. O BarberPro implementa medidas importantes:

- **Hashing de Senhas (BCrypt):** As senhas dos usuários não são armazenadas em texto claro, mas sim como hashes gerados pelo algoritmo BCrypt. Isso protege as credenciais dos usuários mesmo em caso de uma violação de dados, pois é extremamente difícil reverter um hash para a senha original. [1]
- **JSON Web Tokens (JWT):** A autenticação é baseada em JWTs, que são tokens autocontidos e assinados digitalmente. Após o login, o servidor emite um JWT que o cliente armazena e envia em cada requisição subsequente. O servidor valida a assinatura do token para garantir sua autenticidade e integridade, sem a necessidade de consultar o banco de dados a cada requisição. Isso melhora a performance e a escalabilidade. [2]
- **CORS (Cross-Origin Resource Sharing):** A configuração de CORS no backend é fundamental para a segurança. Ela define quais domínios (origens) têm permissão para acessar a API. Ao restringir as origens a `https://barberproapp.netlify.app`, `http://localhost:3000` e `http://localhost:5173`, o sistema se protege contra ataques de Cross-Site Request Forgery (CSRF) e outras vulnerabilidades relacionadas a requisições de origens não autorizadas. [3]
- **Autenticação Google:** A integração com a autenticação via Google adiciona uma camada de conveniência e segurança, aproveitando a infraestrutura de segurança do Google para gerenciar as credenciais do usuário. O backend verifica a validade do `id_token` do Google, garantindo que apenas usuários autenticados pelo Google possam acessar o sistema por essa via.

10.3. Gerenciamento de Banco de Dados com Entity Framework Core e Migrações

A escolha do PostgreSQL em conjunto com o Entity Framework Core (EF Core) e o uso de migrações é um ponto forte para a gestão do banco de dados:

- **ORM (Object-Relational Mapper):** O EF Core abstrai a complexidade das operações SQL, permitindo que os desenvolvedores interajam com o banco de dados usando objetos C#. Isso acelera o desenvolvimento e reduz a chance de erros relacionados a SQL. [4]
- **Migrações Automáticas:** A aplicação automática das migrações do EF Core na inicialização (`dbContext.Database.Migrate()`) garante que o esquema do banco de dados esteja sempre atualizado com o modelo de dados da aplicação. Isso é particularmente útil em ambientes de desenvolvimento e CI/CD, mas requer atenção em ambientes de produção para evitar interrupções.
- **Scripts SQL para Inicialização:** O arquivo `init.sql` no diretório `Database/` é essencial para a configuração inicial do banco de dados, incluindo a criação de tabelas, índices, funções e triggers. A função `gerar_codigo_convite()` e os triggers para `data_atualizacao` demonstram a automação de tarefas de banco de dados, o que contribui para a integridade e consistência dos dados.

10.4. Usabilidade Focada no Usuário

O design da interface do usuário, com dashboards e fluxos de trabalho específicos para Clientes, Barbeiros e Gerentes, é um ponto importante que visa otimizar a experiência de cada perfil. A clareza na navegação, a

gestão de agendamentos e a visualização de informações relevantes para cada papel contribuem para a eficiência e satisfação do usuário.

10.5. Tratamento de Erros e Validações

A implementação de validações de entrada tanto no frontend quanto no backend, juntamente com um tratamento de erros consistente que retorna mensagens claras e códigos de status HTTP apropriados, é crucial para a robustez da aplicação. Isso ajuda a prevenir dados inválidos e a fornecer feedback útil aos usuários e a outros sistemas que consomem a API.

Referências

[1] BCrypt. (n.d.). *Wikipedia*. Retrieved from <https://en.wikipedia.org/wiki/Bcrypt> [2] JSON Web Token (JWT). (n.d.). *jwt.io*. Retrieved from <https://jwt.io/> [3] Cross-Origin Resource Sharing (CORS). (n.d.). *MDN Web Docs*. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS> [4] Entity Framework Core. (n.d.). *Microsoft Learn*. Retrieved from <https://learn.microsoft.com/en-us/ef/core/>

11. Pontos de Atenção

Embora o projeto BarberPro apresente uma arquitetura sólida e boas práticas de desenvolvimento, existem alguns pontos de atenção que merecem consideração para futuras melhorias, otimizações e para garantir a robustez e segurança a longo prazo.

11.1. Segurança e Gerenciamento de Segredos

- **Token JWT no Frontend:** Atualmente, o token JWT é armazenado no frontend (provavelmente no `localStorage` ou `sessionStorage`). Embora seja uma prática comum, o `localStorage` é vulnerável a ataques de Cross-Site Scripting (XSS). Uma alternativa mais segura seria o uso de `HttpOnly` cookies, que não são acessíveis via JavaScript, mitigando o risco de XSS. [5]
- **Chaves de API e Segredos:** As chaves de API (como a `Jwt:Key` e `GoogleAuth:ClientId`) são configuradas via `appsettings.json` ou variáveis de ambiente. Em ambientes de produção, é crucial garantir que esses segredos sejam gerenciados de forma segura, utilizando serviços de gerenciamento de segredos (ex: Azure Key Vault, AWS Secrets Manager, HashiCorp Vault) em vez de armazená-los diretamente no código ou em arquivos de configuração que possam ser expostos. [6]
- **Validação de Entrada no Backend:** Embora haja validação de entrada nos controladores, é importante garantir que todas as entradas do usuário sejam rigorosamente validadas e sanitizadas para prevenir ataques como injeção de SQL, XSS e outros. A validação deve ser abrangente e considerar todos os possíveis vetores de ataque.

11.2. Escalabilidade e Performance

- **Otimização de Consultas SQL:** Para uma aplicação SaaS, o volume de dados e requisições pode crescer rapidamente. É fundamental monitorar e otimizar as consultas SQL geradas pelo Entity Framework Core para garantir que o banco de dados não se torne um gargalo de performance. O uso de índices já é um bom começo, mas análises de performance e otimizações específicas podem ser necessárias.

- **Cache:** Para reduzir a carga no banco de dados e melhorar o tempo de resposta, a implementação de uma camada de cache (ex: Redis) para dados frequentemente acessados (como informações de barbearias ou serviços) pode ser benéfica.
- **Filas de Mensagens:** Para operações que podem ser demoradas ou que não precisam de uma resposta imediata (ex: envio de e-mails de confirmação, processamento de relatórios), a utilização de filas de mensagens (ex: RabbitMQ, Kafka) pode desacoplar o backend e melhorar a responsividade da aplicação. [7]

11.3. Tratamento de Erros e Logs

- **Log Centralizado:** Atualmente, os logs parecem ser escritos em arquivos locais (`backend.log` , `backend_full_log.log`). Em um ambiente de produção, é essencial ter um sistema de log centralizado (ex: ELK Stack, Grafana Loki) que permita coletar, armazenar e analisar logs de todas as instâncias da aplicação. Isso facilita a depuração, o monitoramento e a identificação proativa de problemas. [8]
- **Monitoramento e Alertas:** Implementar ferramentas de monitoramento de performance (APM - Application Performance Monitoring) e alertas para métricas críticas (uso de CPU, memória, erros de requisição, latência) é vital para garantir a disponibilidade e o bom funcionamento do sistema.

11.4. Testes

- **Cobertura de Testes:** A documentação não detalha a estratégia de testes. Para garantir a qualidade e a estabilidade do código, é crucial ter uma boa cobertura de testes unitários (para lógica de negócios e componentes individuais), testes de integração (para verificar a comunicação entre os módulos) e testes end-to-end (para simular o fluxo completo do usuário). [9]

11.5. CI/CD (Integração Contínua/Entrega Contínua)

- **Automação de Deploy:** Embora o projeto possa ser implantado manualmente, a implementação de um pipeline de CI/CD (ex: GitHub Actions, GitLab CI/CD, Jenkins) automatizaria o processo de build, teste e deploy. Isso reduz erros manuais, acelera as entregas e garante consistência entre os ambientes. [10]

11.6. Usabilidade e Experiência do Usuário (UX)

- **Feedback ao Usuário:** Garantir que o frontend forneça feedback claro e imediato ao usuário sobre o status das operações (carregamento, sucesso, erro) é fundamental para uma boa UX. Isso inclui indicadores de carregamento, mensagens de sucesso/erro e validações em tempo real.
- **Acessibilidade:** Avaliar e melhorar a acessibilidade da interface para usuários com deficiência (ex: conformidade com WCAG) pode expandir o alcance da aplicação.

Referências

[5] OWASP Top 10. (n.d.). *OWASP Foundation*. Retrieved from <https://owasp.org/www-project-top-ten/> [6] 12 Factor App. (n.d.). *The Twelve-Factor App*. Retrieved from <https://12factor.net/config> [7] Message Queue. (n.d.). *Wikipedia*. Retrieved from https://en.wikipedia.org/wiki/Message_queue [8] Centralized Logging. (n.d.). *Logz.io*. Retrieved from <https://logz.io/blog/what-is-centralized-logging/> [9] Test Automation. (n.d.). *TechTarget*. Retrieved from <https://www.techtarget.com/searchsoftwarequality/definition/test-automation> [10] CI/CD. (n.d.). *Red Hat*. Retrieved from <https://www.redhat.com/en/topics/devops/what-is-ci-cd>